1 Address Space and Memory-Mapped Registers

The Cortex-M3 sees a single 4 GB physical address space (32-bit addresses). The architecture divides it into regions and fixes the purpose of each one; the **vendor decides how much memory or which devices actually populate them**.

There is **no address translation** (no MMU, hence no virtual memory): the address issued by an instruction is the address seen by memory and peripherals, and no run-time loader chooses where a program is placed. All placement decisions are made at link time and recorded in the image.

1.1 Address Space Overview

Address range	Region	Purpose	Populated on `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	Program memory: code, constants, vector table at `0x0`	4 MB at `0x0` ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	Data memory: variables, heap, stack	4 MB at `0x2000_0000` ("RAM")
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	Registers of the on-chip devices added by the vendor	UARTs, timers, watchdog
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	Memory and devices outside the chip	Not used
`0xE000_0000` – `0xE00F_FFFF`	Private Peripheral Bus	Registers of the core peripherals (NVIC, SysTick, SCB, MPU) and debug units	System Control Space used by the firmware
`0xE010_0000` – `0xFFFF_FFFF`	Vendor-specific	System area reserved for the vendor	Not used in these notes

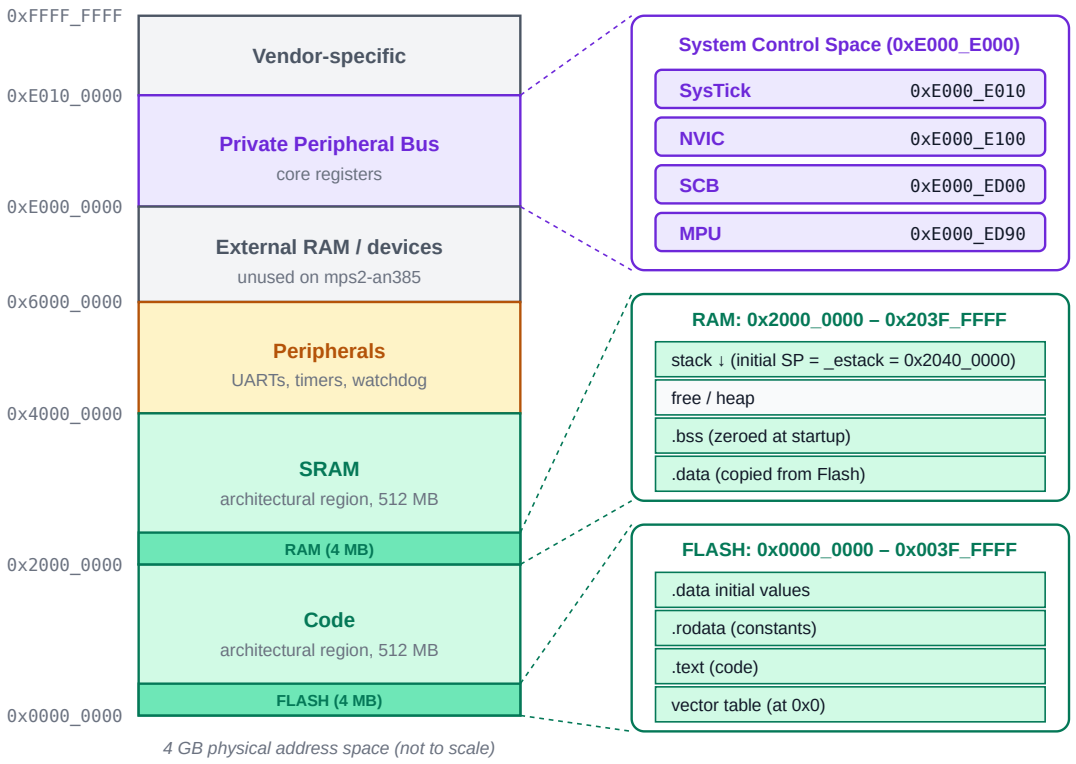


Figure 1.3: *Cortex-M3 address space with the regions populated on mps2-an385 and the System Control Space in detail.*

A region is only a range of addresses: memory or registers exist only where the chip connects something, and an access to an empty address raises a BusFault. On `mps2-an385` only a small part of the 4 GB is populated:



The External RAM / devices and Vendor-specific regions are not used on this board, so the following sections cover only the three populated regions, from the lowest address to the highest.

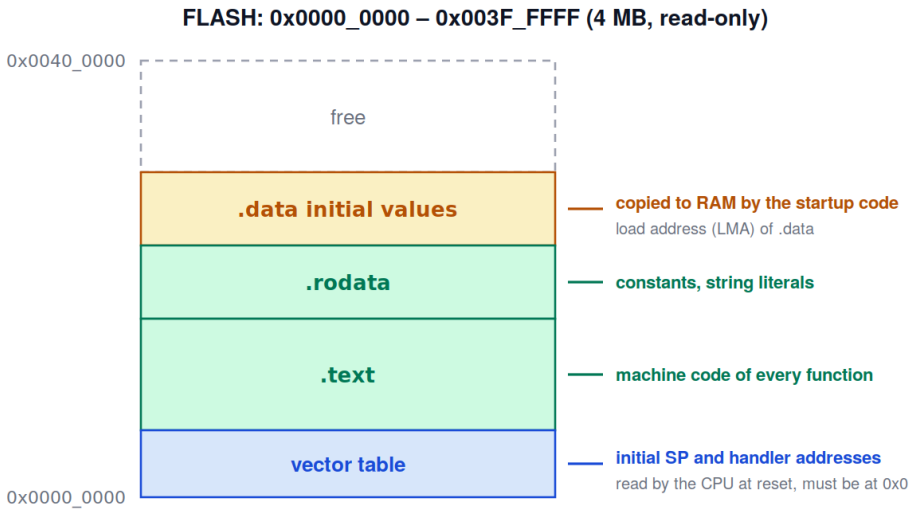
1.2 Code and SRAM: the Memories

The two lowest regions hold the **memories of the chip**. On `mps2-an385`, 4 MB of program memory ("**FLASH**") sit at the start of the Code region, and **4 MB** of data memory ("**RAM**") at the start of the **SRAM region**.

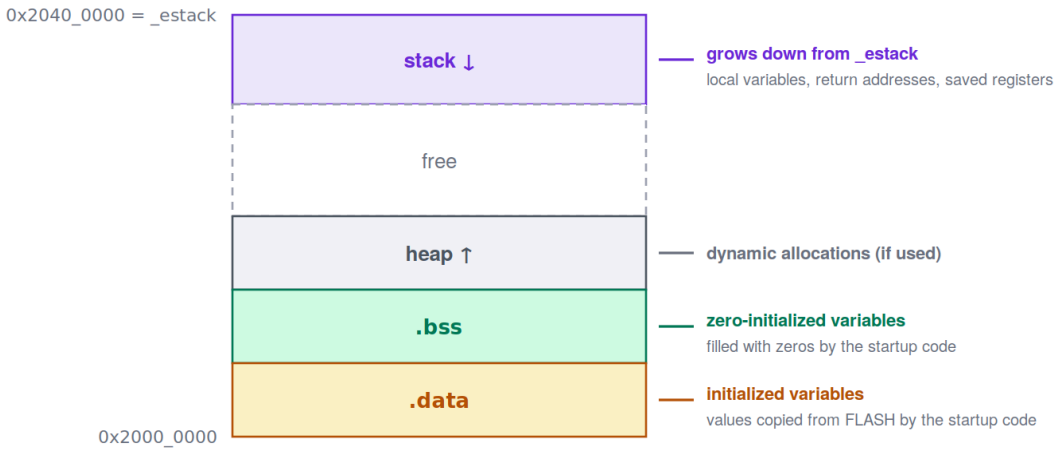
- **Flash** keeps its contents without power. It holds the vector table, the code, the constants, and the initial values of initialized variables.

- **SRAM** is fast writable memory, but its contents are undefined at power-on. It holds the variables (initialized and zeroed), the heap, and the stack.

As a consequence, before C code can run safely, the startup code must copy the initial values of variables from Flash to SRAM and fill the zeroed variables with zeros. Which part of the program goes into which memory is decided by the **linker script**, described in the chapter on boot and CPU configuration.



FLASH contents: *the vector table at address 0x0, the code, the constants, and the initial values of .data.*



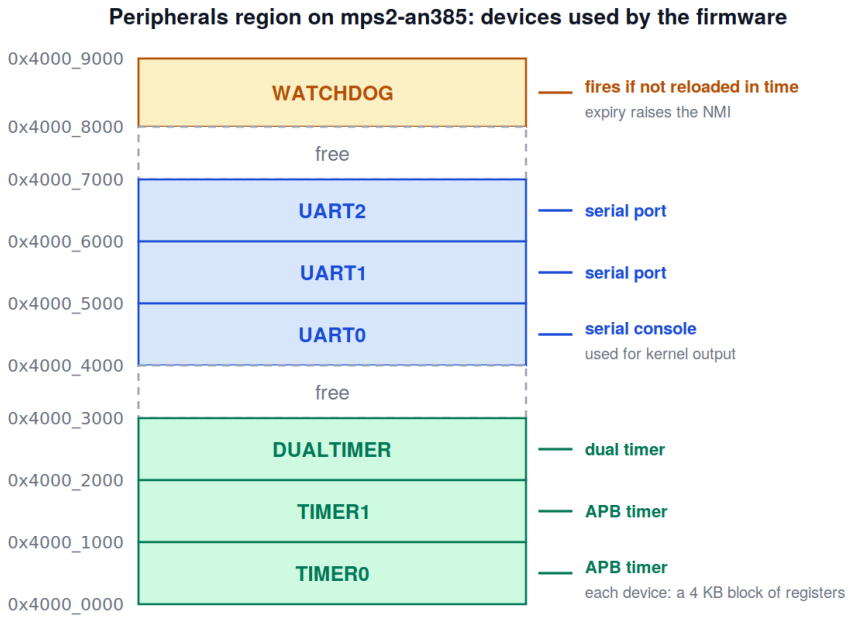
RAM contents: *the variables of .data and .bss at the bottom, the heap above them, and the stack growing down from the top.*

EMULATED FLASH

On `mps2-an385`, and in QEMU, the 4 MB at address `0x0` are fast RAM preloaded with the program image. They play the role of Flash and are treated as read-only, exactly like real Flash on a microcontroller.

1.3 Peripherals: Device Registers

The Peripherals region holds the registers of the devices that the vendor adds around the core: **timers, UARTs, GPIO,** and so on. Each device occupies a small block of addresses, often **4 KB**, and is controlled by reading and writing its registers. Which devices exist, and at which addresses, depends entirely on the chip: on `mps2-an385` they include three UARTs, two timers, a dual timer, and a watchdog, shown in the memory map above. These devices and their drivers are described in the chapter on devices, interrupts, and system calls.

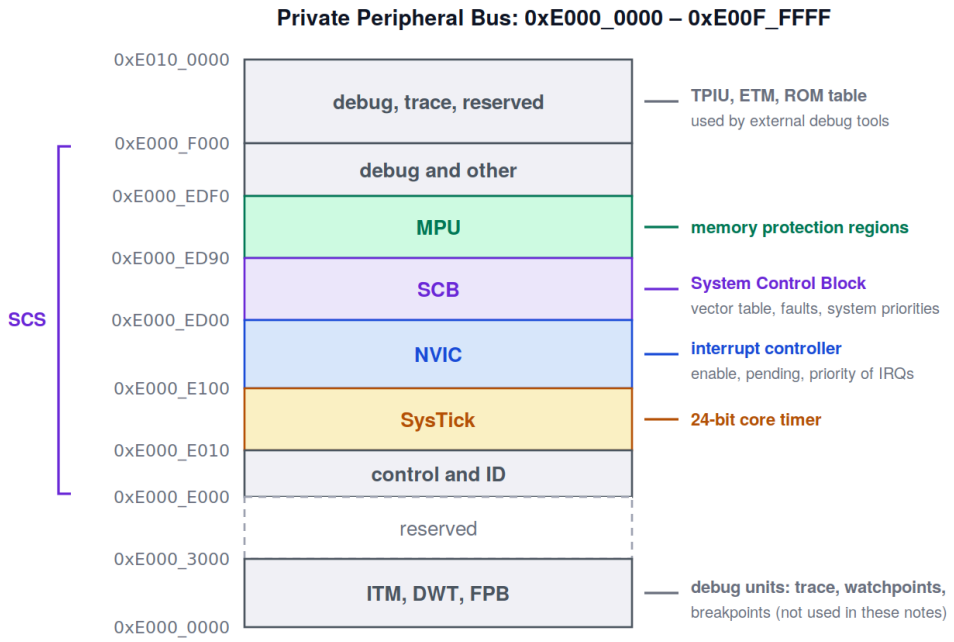


Peripherals contents: *on mps2-an385 each device occupies a 4 KB block of registers; only part of the 512 MB region is used.*

1.4 PPB: the Core Peripherals

The **Private Peripheral Bus** holds the registers of the core peripherals, together with debug and trace units that are not used in these notes. Inside it, the **System Control Space** (SCS, `0xE000_E000` – `0xE000_EFFF`) groups the four core peripherals, which are identical on every Cortex-M3 regardless of the vendor. Their registers are described in the chapter on core peripherals and the MPU.

Core peripheral	Base address	Purpose
SysTick	`0xE000_E010`	24-bit periodic timer
NVIC	`0xE000_E100`	Enabling, pending state, and priority of device interrupts
SCB (System Control Block)	`0xE000_ED00`	Vector table location, fault configuration and status, system priorities
MPU	`0xE000_ED90`	Memory protection regions



PPB contents: *the System Control Space holds the registers of the four core peripherals; the rest of the bus belongs to debug and trace units.*

System Control Space (privileged access only)

0xE000_EFFF

Other 528 B

debug and other registers

0xE000_EDF0

0xE000_EDEF

MPU 96 B

memory protection unit

0xE000_EDA0 ◀ **RASR** region size and permissions

0xE000_ED9C ◀ **RBAR** region base address

0xE000_ED98 ◀ **RNR** region number

0xE000_ED94 ◀ **CTRL** enable, PRIVDEFENA

0xE000_ED90 ◀ **TYPE** number of regions

0xE000_ED90

0xE000_ED8F

SCB 144 B

System Control Block

0xE000_ED38 ◀ **BFAR** BusFault address

0xE000_ED34 ◀ **MMFAR** MemManage fault address

0xE000_ED2C ◀ **HFSR** cause of HardFault

0xE000_ED28 ◀ **CFSR** cause of configurable faults

0xE000_ED24 ◀ **SHCSR** enable fault handlers

0xE000_ED18 ◀ **SHPR1** system exception priorities (SHPR1–3)

0xE000_ED14 ◀ **CCR** fault traps, stack alignment

0xE000_ED0C ◀ **AIRCR** reset, priority grouping

0xE000_ED08 ◀ **VTOR** vector table address

0xE000_ED04 ◀ **ICSR** pending state, PendSV trigger

0xE000_ED00 ◀ **CPUID** core identification

0xE000_ED00

0xE000_ECFF

NVIC 3 KiB

interrupt controller

0xE000_E400 ◀ **IPR** priority, 1 byte per IRQ

0xE000_E300 ◀ **IABR** active (handler running)

0xE000_E280 ◀ **ICPR** clear-pending

0xE000_E200 ◀ **ISPR** set-pending

0xE000_E180 ◀ **ICER** clear-enable

0xE000_E100 ◀ **ISER** set-enable, 1 bit per IRQ

0xE000_E100

0xE000_E0FF

SysTick 240 B

24-bit core timer

0xE000_E01C ◀ **CALIB** calibration value

0xE000_E018 ◀ **VAL** current count

0xE000_E014 ◀ **LOAD** reload value

0xE000_E010 ◀ **CTRL** enable, interrupt, clock source

0xE000_E010

0xE000_E00F

Control 16 B

control and ID registers

0xE000_E000

ONLY PRIVILEGED CODE MAY ACCESS THE SCS.

An unprivileged access raises a BusFault, which is what prevents application code from reconfiguring the machine.